

人工智能实践-数字经济应用实践 系统开发与部署篇

C O U R S E T I T L E

主讲：高刚毅

邮箱：ggy0222@sina.com

部门：信智学院 人工智能系 9-411

智能系统开发流程



Content

01 | Web 开发概述

02 | 系统测试

03 | 模型部署

04 | 职业发展

05 | 阶段考核

1、Web 开发概述

- Web 开发常用模式：

1. B/S (Browser/Server) : 用户通过浏览器访问 Web 应用，所有业务逻辑由服务器处理，数据通过 HTTP 协议传输。

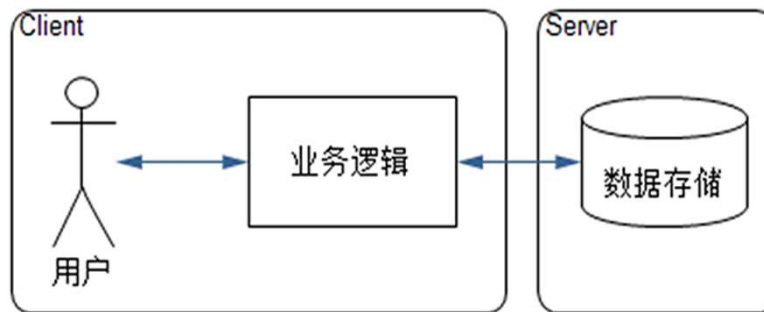
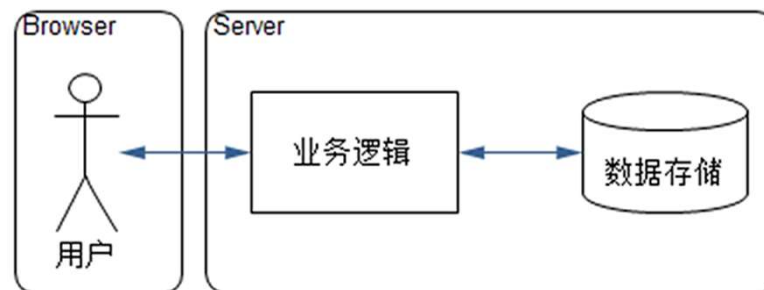
- 特点：零客户端安装；集中式维护；性能依赖网络

2. C/S (Client/Server) : 客户端（桌面应用）直接与服务器通信，客户端承担部分业务逻辑和数据处理任务。

- 特点：客户端需安装；性能优势；维护成本高

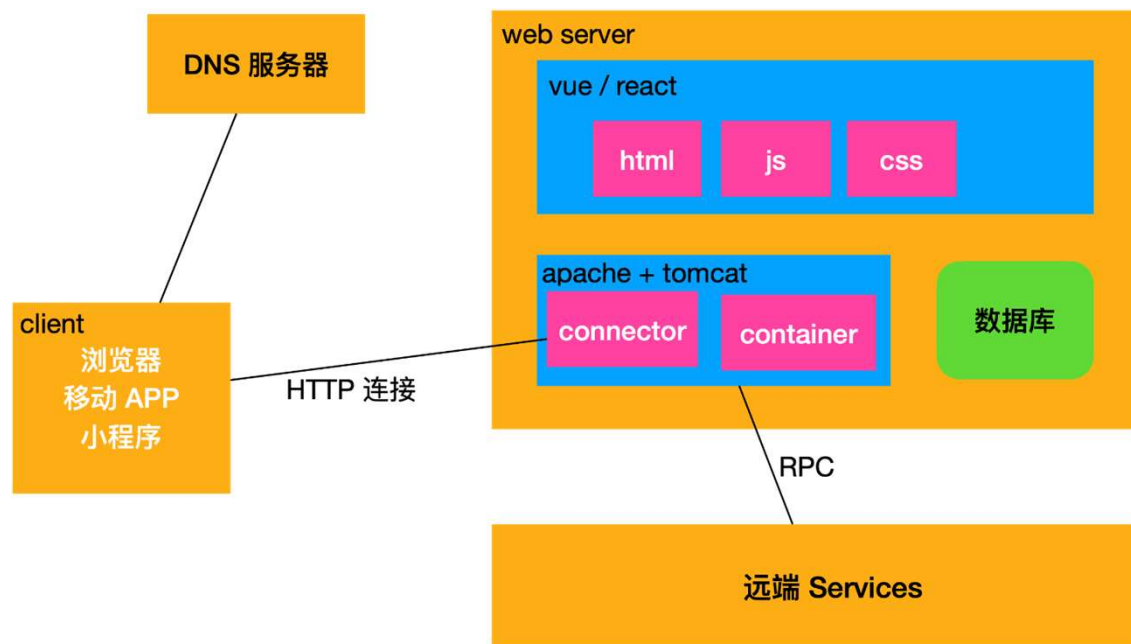
3. 移动开发：通过便携终端进行交互

- 特点：交互方便；功能丰富



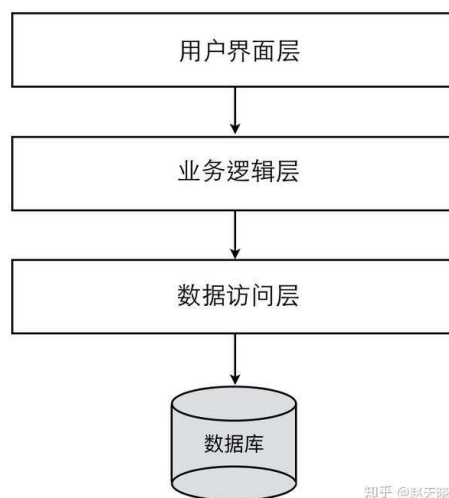
Web 前端与后端

- **前后端分离模式**：前端（浏览器/框架）独立开发界面，后端（API 接口）专注数据处理。
- **Web 前端**：用户直接交互的界面部分，包括网页的视觉设计、布局、动态效果等（如按钮、表单等）。
 - **核心技术**：HTML、CSS、Javascript、框架
- **Web 后端**：处理服务器端逻辑，包括数据存储、业务规则、安全验证等，用户无法直接感知其运行过程。
 - **核心技术**：编程语言、数据库、框架、通信



Web 架构

Web 架构	场景需求
1、 分层架构	应用广泛，适合中小型项目
2、 MVC 架构	表现层的细化实现
3、 微服务架构	大型复杂系统
4、 分布式架构	实时分布式数据处理



- **分层架构** (Layered Architecture)

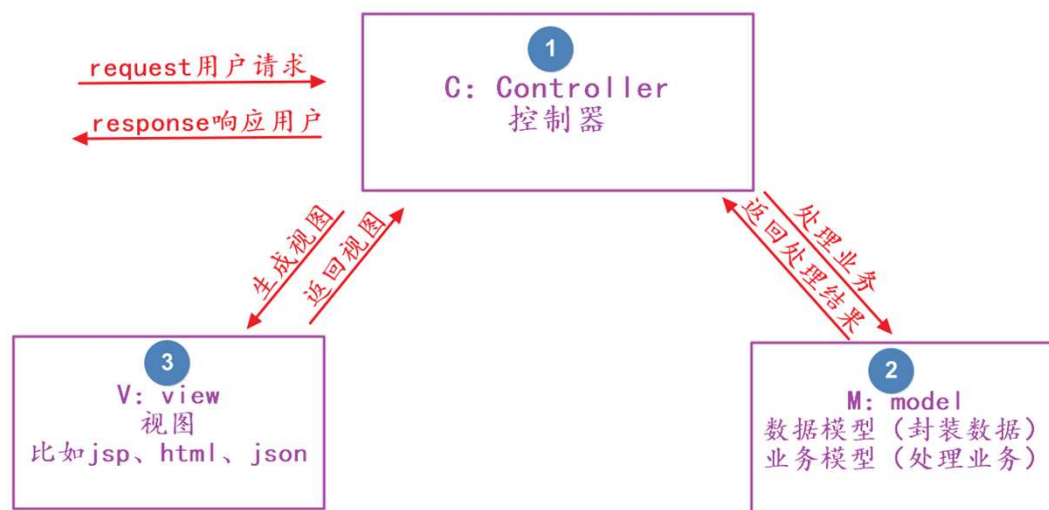
- **表现层** (Presentation Layer)：负责用户交互与数据展示，采用 HTML/CSS/JavaScript 或前端框架（如 React/Vue）实现。
- **业务逻辑层** (Business Logic Layer)：处理核心业务规则，例如用户权限验证、订单处理逻辑等。
- **数据访问层** (Data Access Layer)：提供统一的数据读写接口，通过 ORM 框架（如 Hibernate）或 DAO 模式实现。

- **Web架构**是对Web系统的整体结构划分，核心是通过拆分职责、定义模块间关系，来满足可维护性、可扩展性等需求的设计方案。

Web 架构

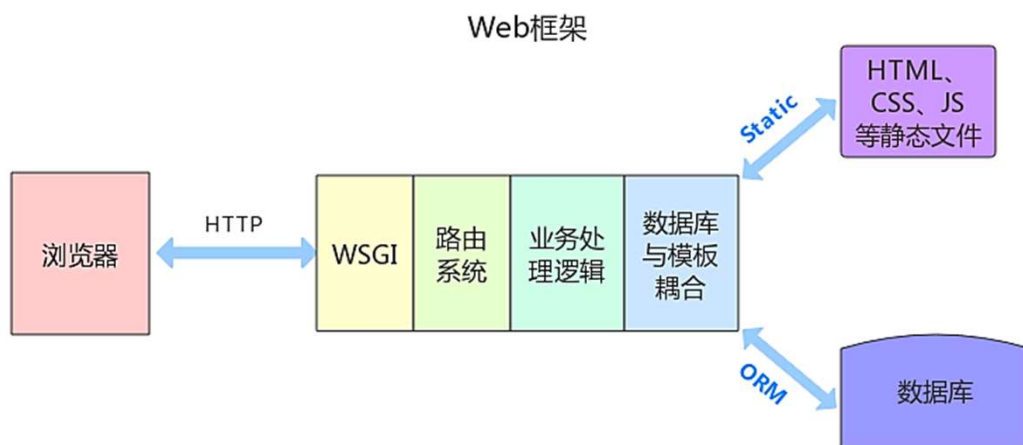
- **MVC 架构** (Model-View-Controller) :
 - **Model** (模型) : 负责**数据管理**和**业务逻辑**实现 (如数据库操作、计算逻辑) 。
 - **View** (视图) : **用户界面**展示 (如 JSP 模板) 。
 - **Controller** (控制器) : 接收用户输入并协调**模型与视图的交互**, 包括解析请求、调用模型处理业务逻辑、更新视图等。
- **典型应用**:
 - **Web 开发**: Spring MVC 框架; Django。
 - **移动应用**: iOS 开发。
 - **桌面应用**: C# 的 WinForms 或 JavaFX。

MVC的设计理念



Web 开发框架

- **Web 开发框架** (Web Framework) 是一套**预先封装好的工具库和代码结构**，旨在**简化** Web 应用的开发流程。框架通常分为**后端框架**和**前端框架**，支持动态网站、网络服务和跨平台应用的快速搭建与部署。
 1. 封装好了大量代码，协助程序**快速开发**，相当于**项目半成品**。
 2. 开发者只需**按照框架约定要求**，在**指定位置**写上自己的**业务逻辑代码**。
- **Web 框架的作用**：
 - 降低开发难度，**提高开发效率**，减少重复编码，使开发者更专注于业务逻辑实现。



常用 Web 前端框架

1. React

- **特点**: 由 Facebook 维护, 基于**组件化**开发模式, 通过虚拟 DOM 提升渲染性能, 支持服务端渲染 (SSR) 和渐进式 Web 应用 (PWA) 开发。
- **适用场景**: 大型SaaS、电商、企业后台。

2. Vue.js

- **特点**: **渐进式框架**, 核心库仅关注视图层, 支持按需集成路由 (Vue Router)、状态管理 (Vuex) 等功能; 体积轻量 (约 30KB), 学习曲线平缓。
- **适用场景**: 中小型项目、快速原型。

3. Angular

- **特点**: 全功能**企业级**框架, 内置依赖注入、模块化系统和 TypeScript 强类型支持, 提供 CLI 工具加速开发流程。
- **适用场景**: **复杂业务逻辑的大型应用**, 如金融、政务、大型后台系统。

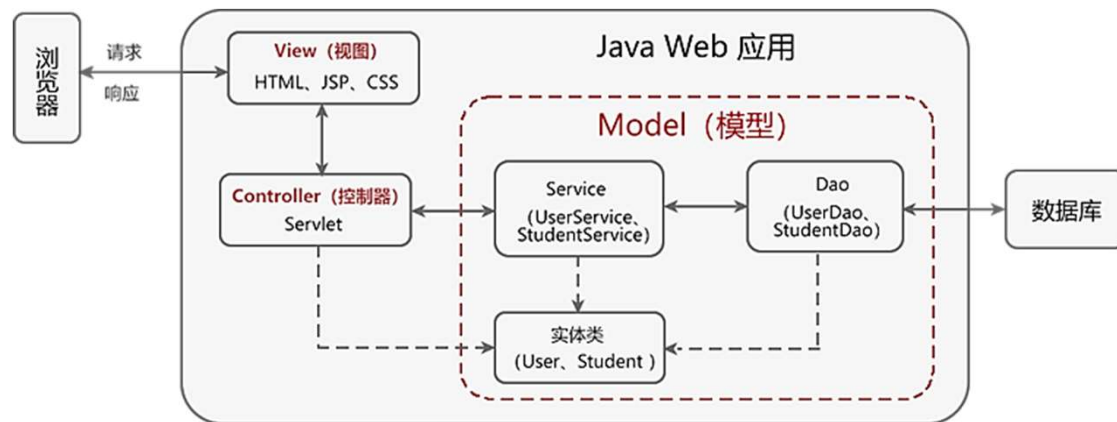
常用 Web 后端框架

- Java 常用 Web 后端框架:

- Spring Boot: 基于 Spring 框架的轻量级快速开发框架, 企业级。
- Hibernate: 对象关系映射框架, 对JDBC进行轻量级对象封装, 数据持久化。
- Struts2: 基于组件的 MVC 模式, 传统企业级。
- JSF: Java EE 标准的 Web 应用框架, 支持 MVC 模式。

- Python 常用 Web 后端框架:

- Django: 完整功能, 重量级, 企业级。
- Tornado: 轻量级, 速度快, 高并发。
- Flask: 提供基本功能, 轻量级, 小型。



2、系统测试

- **软件系统测试**：是针对已集成的完整软件系统进行的验证活动，目的是确保系统在实际运行环境中满足功能、性能、安全等需求。
- **按方法论分类**：
 - **黑盒测试**：基于输入输出验证功能，不关注内部实现（如等价类划分、边界值分析）。
 - **白盒测试**：通过代码逻辑分析覆盖路径、分支和语句（如语句覆盖、路径覆盖）。
 - **灰盒测试**：结合黑盒与白盒方法，部分了解系统内部结构进行测试。

按目标分类	核心内容
功能测试	验证系统功能是否符合需求规格，覆盖正常用例和异常场景。
性能测试	评估系统响应时间、吞吐量等性能指标，包括负载测试和压力测试。
安全测试	检测系统漏洞（如 SQL 注入、XSS 攻击），确保数据保护和权限控制。
兼容性测试	验证系统在不同操作系统、浏览器或硬件环境下的适配能力。
可靠性测试	测试系统在异常条件下的容错能力（如断电恢复、故障转移）。

案例解析：用户登录功能测试

- 一、功能测试
 - 正常登录验证：用例：输入正确的用户名和密码，验证是否跳转至主页且会话建立成功。
 - 异常场景验证：
 - 无效输入验证；输入含有特殊字符验证；空值校验；验证码功能验证。
- 二、安全性测试
 - 密码安全：密码输入时是否加密显示（如星号或圆点）；密码传输是否使用加密协议（如 HTTPS）。
 - 暴力破解：连续多次输入错误密码后，验证账户锁定或延迟登录机制。
- 三、性能测试
 - 响应时间：单用户登录请求的响应时间是否符合预期（如 ≤ 2 秒）。并发性能：模拟 1000 用户同时登录。
- 四、兼容性测试
 - 浏览器适配：主流浏览器显示是否一致；设备适配：在移动端和 PC 端验证页面布局及功能完整性。
- 五、UI 测试
 - 布局与交互：输入框、按钮是否对齐，界面风格是否符合设计规范。默认焦点定位和快捷键是否生效。
- 六、边界条件测试
 - 密码长度限制：输入密码长度等于最小限制（如 6 位）或最大限制（如 18 位），验证是否允许登录。
 - 验证码时效性：验证码在有效期内输入有效，超时后输入失效。

3、模型部署

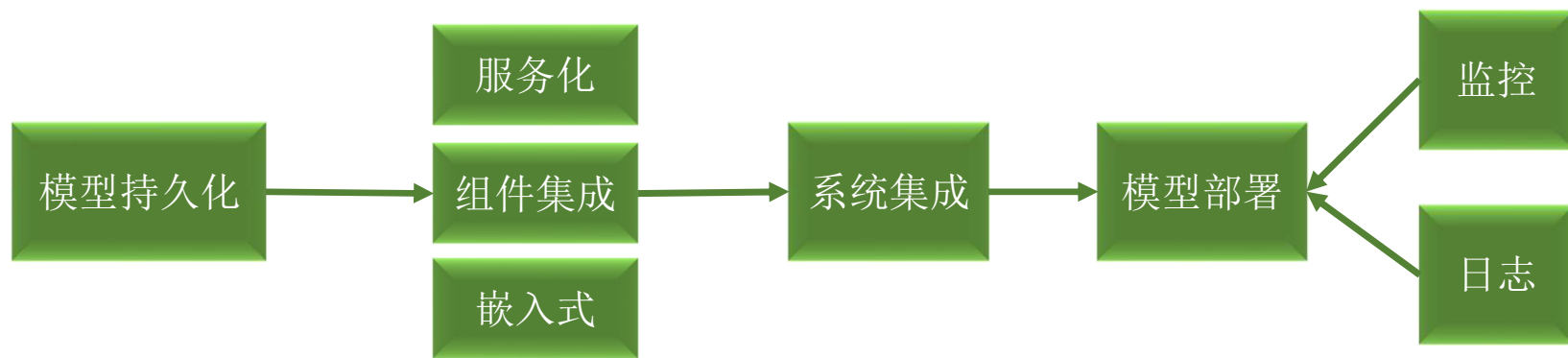
- **模型部署**：是将训练好的机器学习/深度学习模型集成至实际生产环境的过程，确保其能够稳定、高效地处理真实数据并输出预测结果。
- **核心目标包括**：
 1. **功能可用性**：模型在生产环境中按预期执行任务（如分类、生成）。
 2. **性能保障**：满足实时性（低延迟）、吞吐量（高并发）和资源利用率要求。
 3. **持续迭代**：支持模型更新与在线学习，适应数据分布变化。
- **模型部署的核心挑战**：
 1. **环境适配与异构支持**：硬件差异，需适配不同计算单元（如 CPU、GPU、NPU）和指令集，优化计算效率。框架依赖，跨机器/深度学习框架的模型转换与兼容性问题。
 2. **性能瓶颈**：计算资源需求，大模型参数量导致内存占用高，推理延迟显著增加。访存限制，显存带宽不足时，GPU 计算能力无法充分利用，形成性能瓶颈。
 3. **数据一致性与可靠性**：训练-服务偏差；实时监控缺失。
 4. **可扩展性与容错性**：动态扩缩容；故障恢复。

3、模型部署

- 主流部署方法：
 - API服务 + Docker：使用FastAPI/Flask封装模型，打包为Docker镜像，部署于虚拟机或容器平台。适合中小型应用。
 - Kubernetes + KServe：基于K8s的云原生推理平台，支持多模型、自动扩缩容。企业级AI平台、多模型并行。
 - 专用推理服务器：使用TorchServe、TensorFlow Serving等专用框架。适合高性能、低延迟推理场景。
 - 边缘设备部署：在终端设备（如手机、IoT）部署轻量模型，降低云端依赖。

模型优化方法	作用
量化	降低模型精度（如 FP32 → INT8），减少内存占用和计算耗时。
剪枝	移除冗余参数或神经元，压缩模型体积，提升推理速度。
蒸馏	用轻量级模型模仿大模型行为，平衡性能与资源消耗。

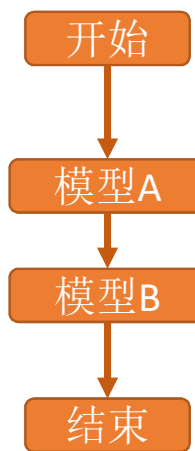
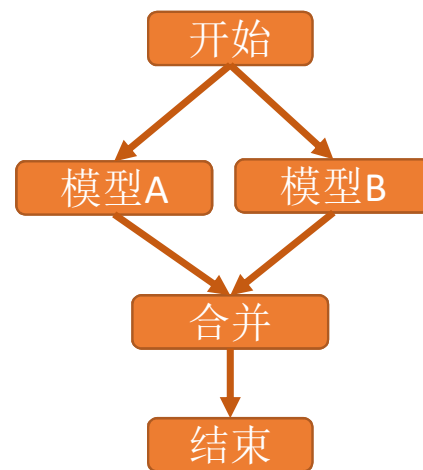
模型部署过程



单模型处理



复合模型处理 构成DAG



容器化



容器

一种轻量级虚拟化技术，通过操作系统层面的资源隔离和共享机制，将应用程序及其依赖（代码、文件、环境变量等）打包为独立、可移植的标准化单元。



容器的作用

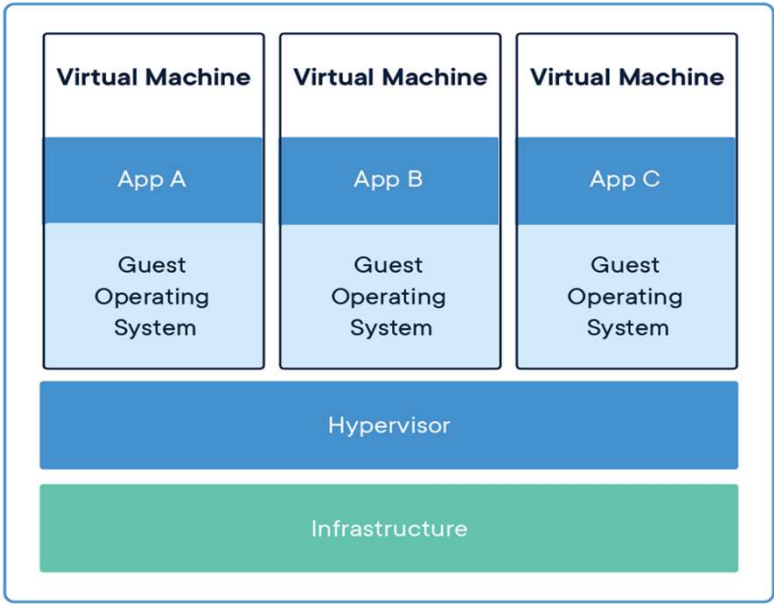
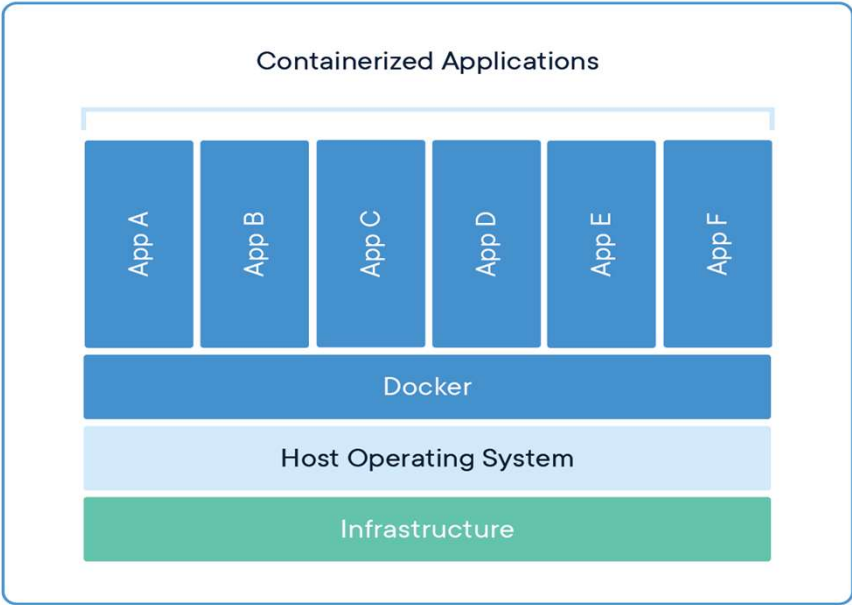
- 环境一致性保障
- 快速部署与扩展
- 资源高效利用
- 跨平台可移植性
- 安全性与隔离性



主流容器实例

- **Docker**：提供标准化的容器镜像格式（如 Dockerfile）和运行引擎。轻量化容器引擎，跨平台可移植，开发友好，解决了环境一致性与资源效率问题。
- **Kubernetes**（K8S）：规模化容器编排，自动化部署与灰度发布，镜像构建，可应对复杂生产环境的稳定性与扩展性需求。

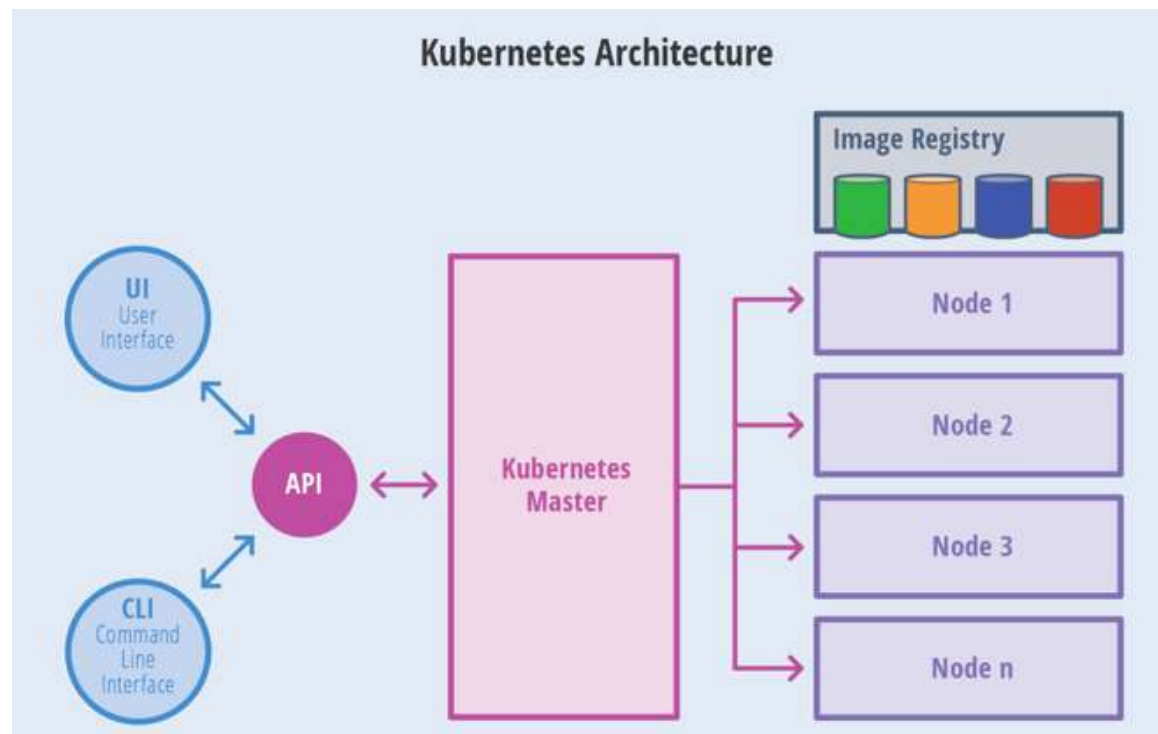
容器与虚拟机



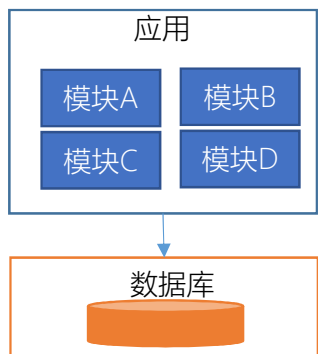
特性	容器	虚拟机 (VM)
虚拟化层级	操作系统级虚拟化，共享宿主机内核	硬件级虚拟化，通过Hypervisor (如VMware、KVM) 模拟完整硬件环境
运行单元	进程级隔离，容器内为应用进程+依赖	完整操作系统实例 (Guest OS) +应用+依赖库
资源占用	无需独立操作系统，镜像体积小 (MB级)	包含完整操作系统，镜像体积大 (GB级)

Kubernetes

- **Kubernetes** (简称 k8s) 是一个**开源**的，用于管理**云平台**中多个主机上的容器化的应用。目标是让部署容器化的应用**简单并且高效**，Kubernetes 提供了应用部署，规划，更新，维护的一种机制。
 - **可移植**：支持公有云，私有云，混合云，多重云 (multi-cloud)
 - **可扩展**：模块化, 插件化, 可挂载, 可组合
 - **自动化**：自动部署，自动重启，自动复制，自动伸缩/扩展

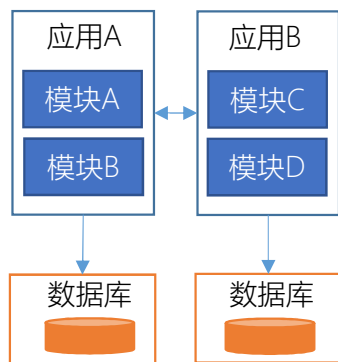


服务演进



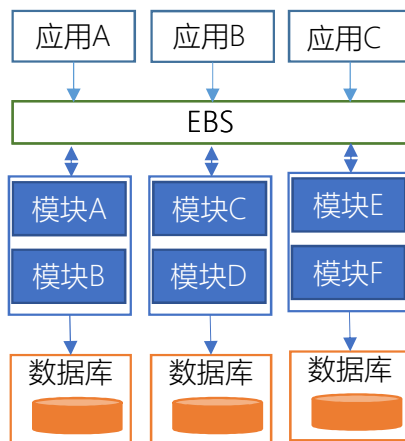
1 单体架构

- 所有的功能集成在一个项目工程中
- 应用与数据库分开部署
- 通过部署应用集群和数据库集群来提高系统的性能



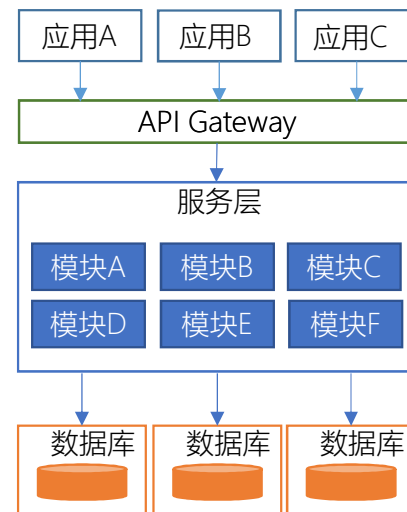
2 垂直架构

- 以单体结构规模的项目为单位进行垂直划分项目
- 项目与项目之间的存在数据冗余，耦合性较大
- 项目之间的接口多为数据同步功能



3 SOA架构

- 将重复公用的功能抽取为组件，以服务的方式给各系统提供服务。
- 各各项目（系统）与服务之间采用webservice、rpc等方式进行通信。
- ESB企业服务总线作为项目与服务之间通信的桥梁。



4 微服务架构

- 将系统服务层完全独立出来，并将服务层抽取为一个一个的微服务
- 微服务遵循单一原则
- 微服务之间采用RESTful等轻量协议传输

服务通信

HTTP	WebSocket
应用层的面向对象的协议	在单个TCP连接上进行全双工通讯的协议
无连接	持久性的连接
无状态	有状态
单向，等待客户端发起请求服务端才能响应	双向，允许服务端主动向客户端推送数据
延迟较高	低延迟
每次请求要携带完整的头部	在连接创建后，用于协议控制的数据包头部相对较小
无连接，高并发，占用系统资源少	长连接，低并发，占用系统资源多

REST	RPC
基于HTTP的一种 Web Service	通过网络从远程计算机程序上请求服务，RPC跨越了传输层和应用层
以资源为中心	以动作为中心
B/S	C/S
Json/XML	自定义结构
可读性好，易扩展	可读性差，不易扩展
松耦合，HTTP标准协议，客户端丰富	高耦合，客户端与采用的框架有关
通信效率低，性能差	通信效率高，性能高
适于外部接口	适于内部接口

4、职业发展

01

大数据工程师

职责涵盖 数据采集、存储、处理、分析及治理等全流程，核心技能包括：数据采集和预处理、大数据框架、分布式系统原理

02

算法工程师

推动智能化落地的核心岗位，核心技能包括：算法与模型能力（机器学习/深度学习）、工程实践能力等

03

AI 部署工程师

连接算法研发与产品落地的关键岗位，核心技能包括：模型转换与压缩、推理优化、异构平台适配、容器化部署等

04

AI 架构师

统筹AI系统全生命周期设计的战略级技术专家，核心技能包括：AI框架与工具链、系统分层设计、服务治理能力、性能调优等



5、阶段考核

- **阶段目标**：开发并实现智能保险风控系统，并将训练好的机器学习模型部署到系统，能够实现对样本数据的预测功能，界面友好，运行稳定。
- **系统主要功能**：
 - 数据读取——》数据预处理与特征工程——》欺诈预测
 - ——》审核审批——》数据归档
 - 用户管理；数据分析与可视化等
- **测试文件**：test.xlsx。
- **答辩展示**：请自带笔记本电脑，按处理流程操作演示。

The image features a dark blue background with a complex, glowing circuit board pattern. In the center, there is a dark blue, rounded cloud-like shape. Inside this shape, the Chinese text "谢谢观看!" (Thank you for watching!) is written in a bold, white, sans-serif font. Several bright blue, glowing lines, resembling electrical traces or data streams, enter the cloud from the top and exit from the bottom, creating a sense of connectivity and flow.

谢谢观看!